



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树



树形查找

二叉排序树BST
(二叉搜索树)

平衡二叉树AVL

红黑树

- 性质：
- 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值。
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值。
 - 它的左右子树也分别为二叉排序树

中序遍历一棵二叉树时可以得到一个结点值递增的有序序列



- 性质：
- 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值。
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值。
 - 它的左右子树也分别为二叉排序树

中序遍历一棵二叉树时可以得到一个结点值递增的有序序列

查找规则

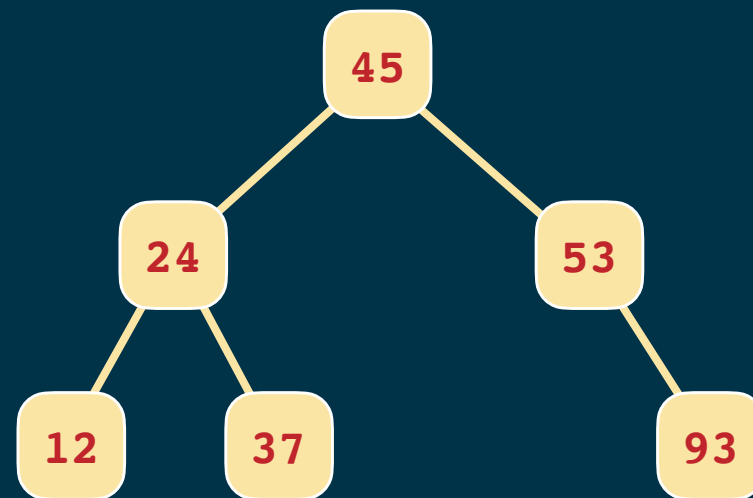
若二叉排序树为空，则查找失败，返回空指针

若二叉排序树非空，将给定值key与根节点的关键字T->data.key进行比较

若key等于T->data.key，则查找成功，返回根节点地址

若key小于T->data.key，则递归查找左子树

若key大于T->data.key，则递归查找右子树



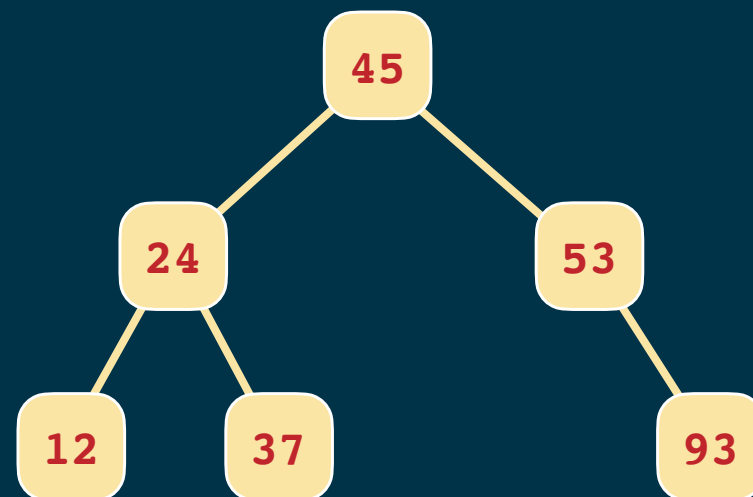


- 性质：
- 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值。
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值。
 - 它的左右子树也分别为二叉排序树

中序遍历一棵二叉树时可以得到一个结点值递增的有序序列

```
BSTree SearchBST (BSTree T, KeyType key)
{ //查找成功返回该数据元素节点的指针，否则返回空指针
  if( (!T) || key==T->data.key )
    return T;
  else if (key<T->data.key)
    //在左子树中继续查找
    return SearchBST (T->lchild, key) ;
  else
    //在右子树中继续查找
    return SearchBST(T->rchild, key) ;
}
```

查找元素(失败情况)





ASL计算

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

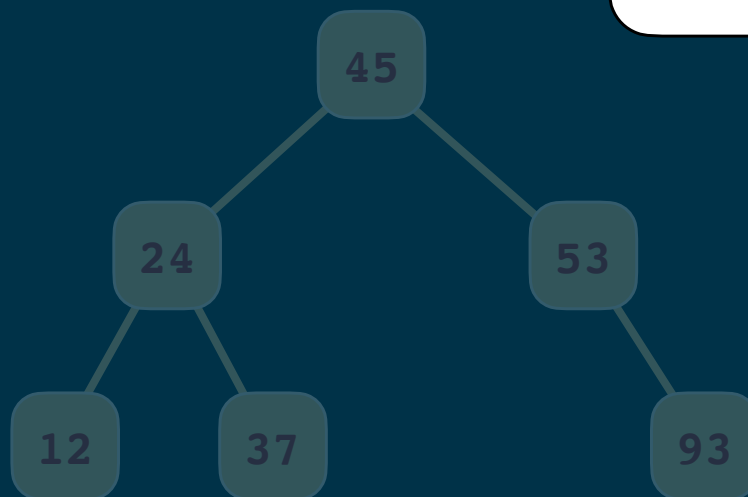
人话：比较次数的期望

$$\sum_{i=1}^n p_i c_i$$

p_i ：每个元素被查找的概率

c_i ：对应元素被查的时候，会进行比较的次数

乘起来求和就是所求期望





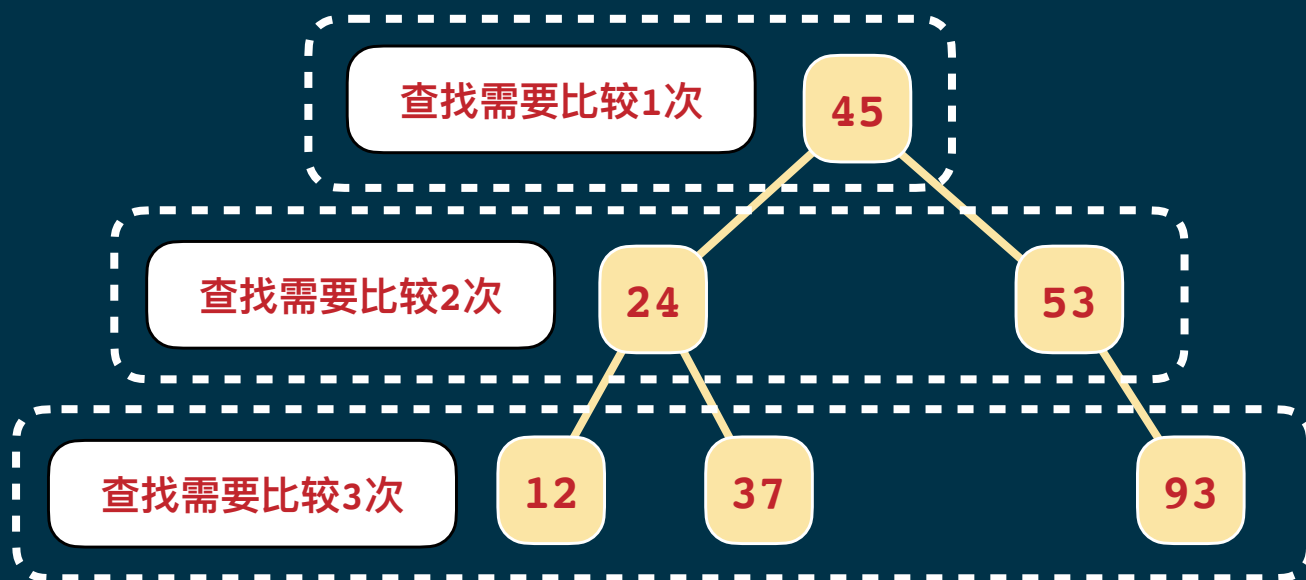
ASL计算

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

人话：比较次数的期望

$$\sum_{i=1}^n p_i c_i$$

共6个元素，题目没说默认等概率查找
每个元素被查概率 $p_i = \frac{1}{6}$



$$ASL = \frac{1}{6}(1 + 2 + 2 + 3 + 3 + 3) = \frac{14}{6} = \frac{7}{3}$$



ASL计算

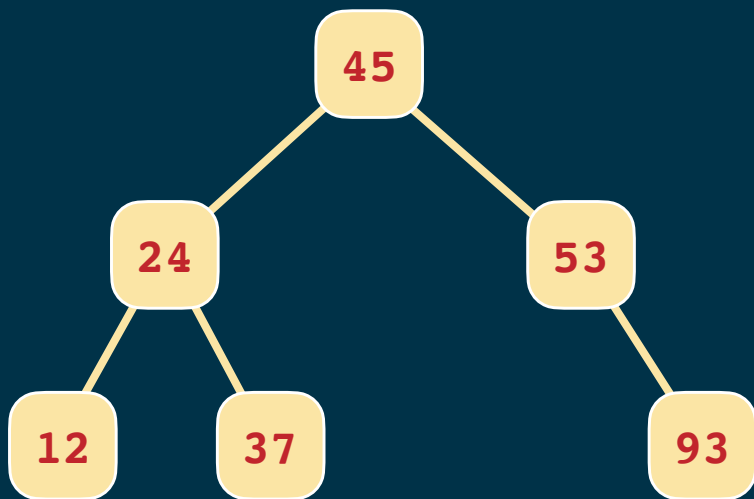
平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

人话：比较次数的期望

共6个元素，题目没说默认等概率查找

树型均衡，左右子树高度之差的绝对值不超过1的二叉排序树BST（被称为平衡二叉树AVL），ASL的复杂度是 $O(\log_2 n)$

$$ASL = \frac{1}{6}(1 + 2 + 2 + 3 + 3 + 3) = \frac{14}{6} = \frac{7}{3}$$



$$\sum_{i=1}^n P_i C_i$$

$$ASL' = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = \frac{7}{2}$$





ASL计算

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

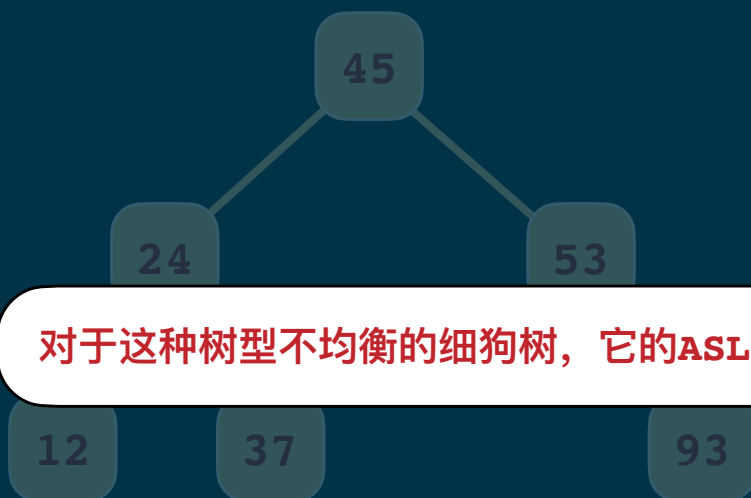
人话：比较次数的期望

$$\sum_{i=1}^n p_i c_i$$

共6个元素，题目没说默认等概率查找

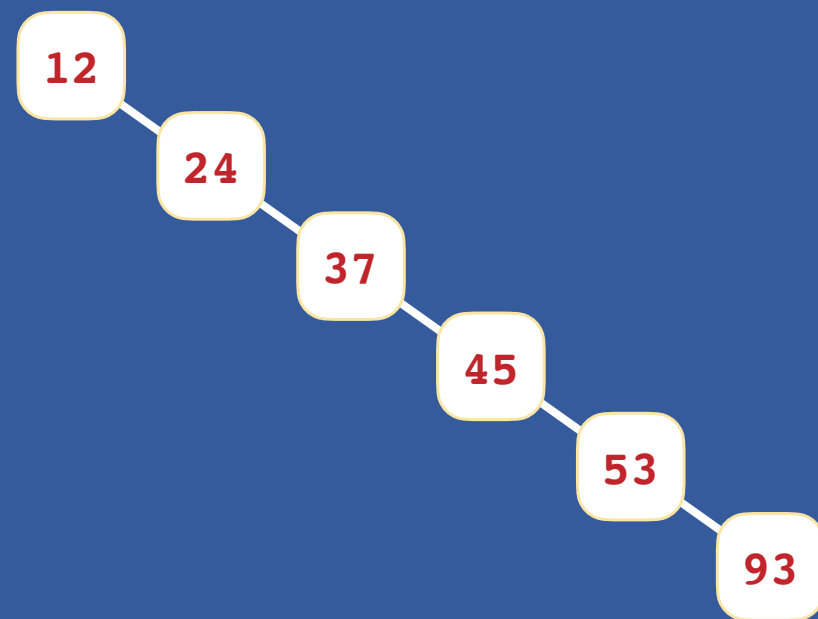
每个元素被查概率 $p_i = \frac{1}{6}$

$$ASL = \frac{1}{6}(1 + 2 + 2 + 3 + 3 + 3) = \frac{14}{6} = \frac{7}{3}$$



对于这种树型不均衡的细狗树，它的ASL数量级(复杂度)是 $O(n)$

$$ASL' = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = \frac{7}{2}$$





树形查找

二叉排序树BST
(二叉搜索树)

平衡二叉树AVL

红黑树

插入操作

插入操作以查找为基础，新插入的节点一定是叶子结点，
并且是查找不成功时查找路径上访问的最后一个节点的左孩子或右孩子节点



插入操作

插入操作以查找为基础，新插入的节点一定是叶子结点，
并且是查找不成功时查找路径上访问的最后一个节点的左孩子或右孩子节点

插入流程

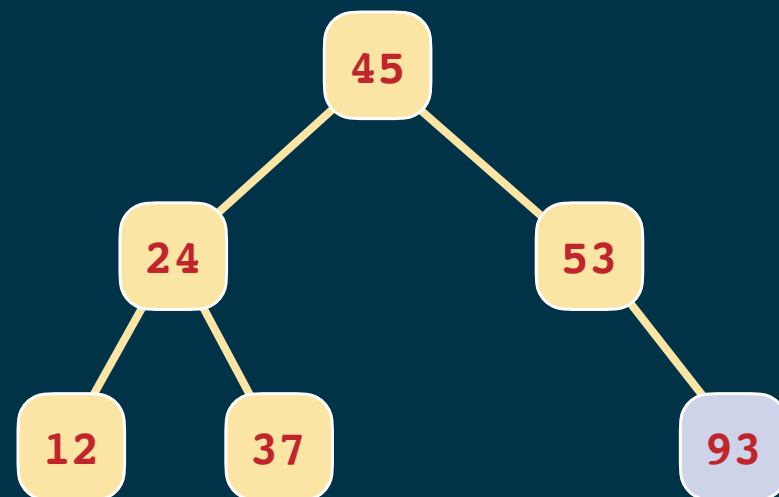
若二叉排序树为空，则将待插入节点*s作为根节点插入空树

若二叉排序树非空，则将key与根节点的关键字T->data.key进行比较：

若key小于T->data.key，则将*s插入左子树

若key大于T->data.key，则将*s插入右子树

插入节点



插入完成



插入操作

插入操作以查找为基础，新插入的节点一定是叶子结点，
并且是查找不成功时查找路径上访问的最后一个节点的左孩子或右孩子节点

```
void InsertBST (BSTree &T, ElemType e){  
    // 如果树是空的，或者找到了应该插入新节点的位置  
    if (T = NULL){  
        // 创建一个新的节点  
        BSTNode *S = new BSTNode;  
        S->data = e;  
        S->lchild = S->rchild = NULL;  
        T = S; // 将新节点链接到树上  
    }  
    else if (e.key < T->data.key)  
        InsertBST(T->lchild, e);  
    else if (e.key > T->data.key)  
        InsertBST(T->rchild, e);  
    // 如果e的键等于当前节点的键，则不进行操作，BST中不允许键值  
    重复
```

插入节点



二叉排序树插入的基本过程是查找，所以时间复杂度同查找一样
 $O(\log_2 n)$

插入完成



树形查找

二叉排序树BST
(二叉搜索树)

平衡二叉树AVL

红黑树

删除操作

删除同样以查找作为基础，删除的为非叶子节点时需要找代替元素上位



删除操作

删除同样以查找为基础，删除的为非叶子节点时需要找代替元素上位

删除流程

若删除的节点为叶子结点：
直接删除

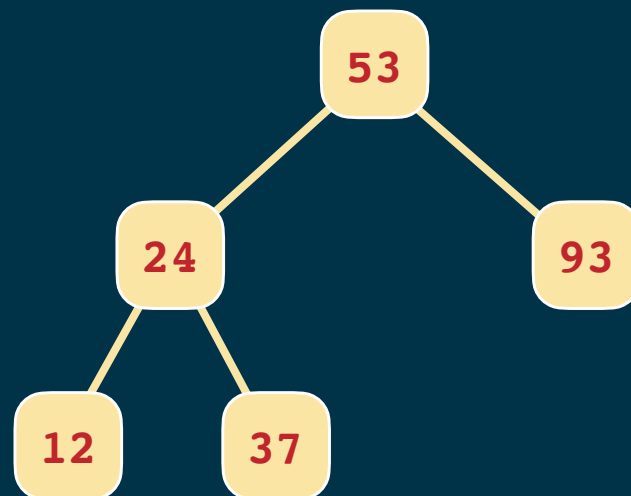
若删除的节点不是叶子结点：
删除节点有左或右孩子
删除后左/右孩子上位；

删除节点有左、右孩子

查找删除节点右子树的最左下孩子，让其与删除节点交换，转换为其他情况

删除节点

45





删除操作

删除同样以查找作为基础，删除的为非叶子节点时需要找代替元素上位

```
void DeleteBST (BSTree &T, KeyType key) {
    BSTree p = T, f = NULL;
    while (p) {
        if (p->data.key == key) break;
        f = p; // 更新父节点
        if (p->data.key > key) p = p->lchild;
        else p = p->rchild; // 向右子树查找
    }
    if (!p) return; // 没有找到目标节点，直接返回
    BSTree q = p;
    if (p->lchild && p->rchild) {
        BSTree s = p->lchild;
        while (s->rchild) { // 寻找最右节点
            q = s; // q指向s的父节点
            s = s->rchild;
        }
        p->data = s->data; // 将s的数据复制到p中
        if (q != p) q->rchild = s->lchild;
        else q->lchild = s->lchild;
        delete s;
    } // if结束
```

```
    else if (!p->rchild) {
        // 无右子树，直接用左子树替换
        p = p->lchild;
    }
    else if (!p->lchild) {
        // 无左子树，直接用右子树替换
        p = p->rchild;
    }
}
```

```
if (!f) T = p; // f为NULL，意味着删除的是根节点
else if (q == f->lchild) f->lchild = p;
// p是f的左子节点
else f->rchild = p; // p是f的右子节点
delete q; // 删除原节点q
}
```

同插入一样，删除的基本操作也是查找，因此时间复杂度仍然是
 $O(\log_2 n)$



树形查找

二叉排序树BST
(二叉搜索树)

平衡二叉树AVL

红黑树

创建操作

如何用一堆元素创建一个二叉排序树



创建操作

如何用一堆元素创建一个二叉排序树

创建流程

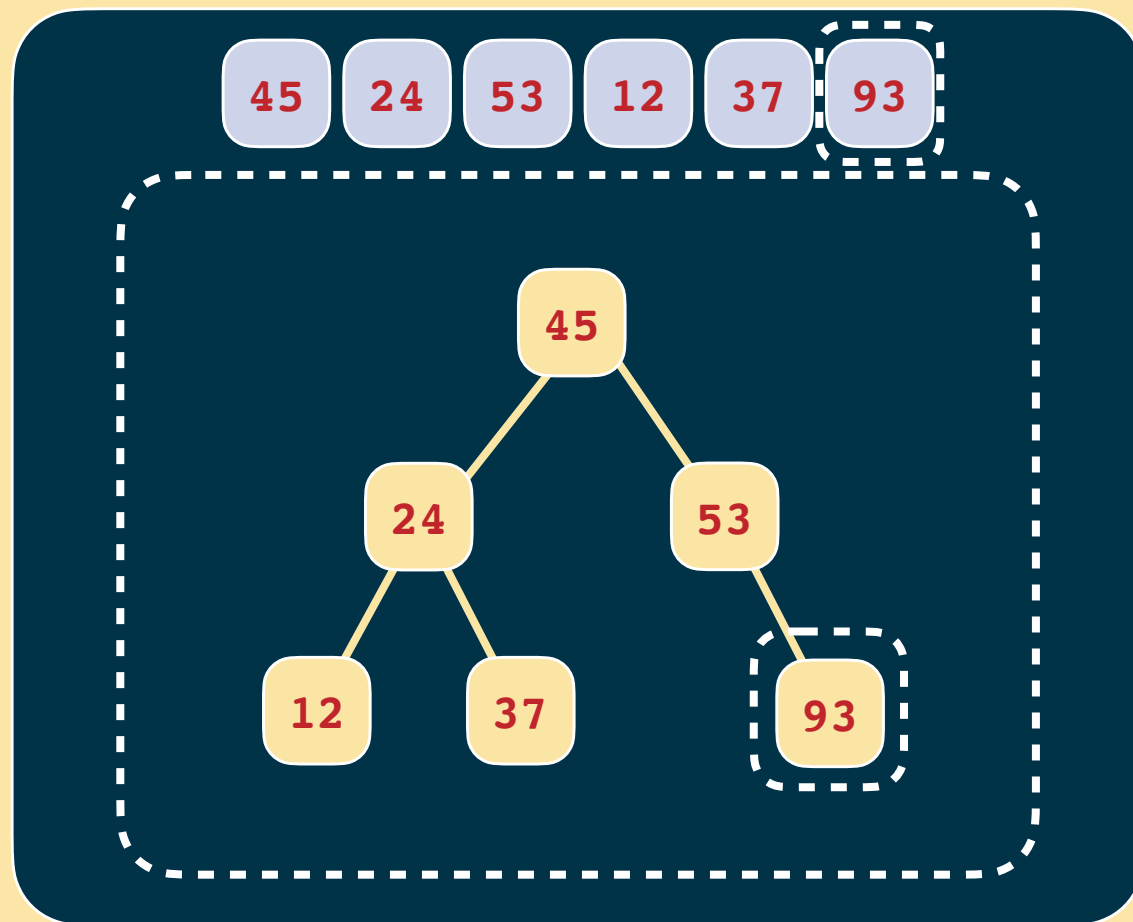
将二叉排序树T初始化为空树

读入一个关键字为key的节点

如果读入的关键字key不是输入结束标志，则循环执行以下操作：

将此节点按照插入的流程插入到二叉排序树T中

读入一个关键字为key的节点

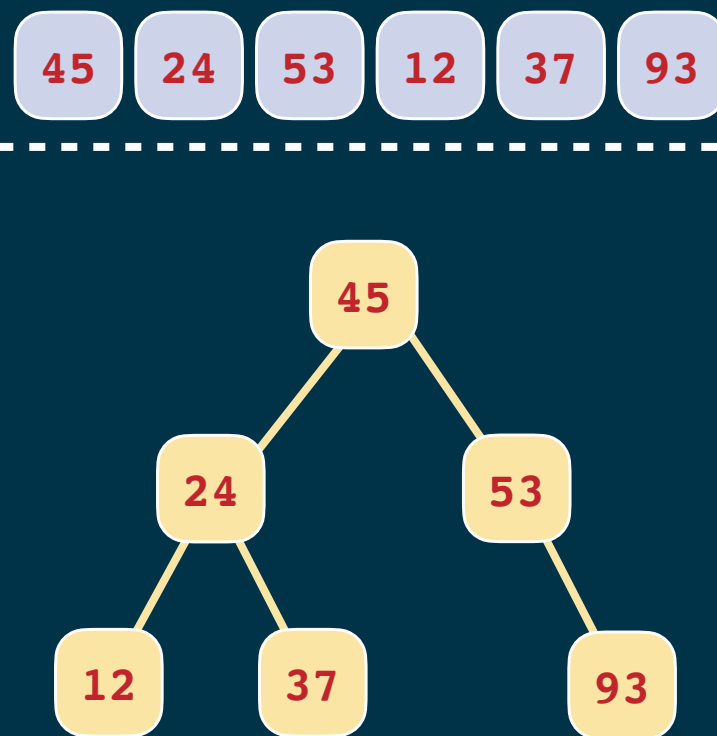




创建操作

如何用一堆元素创建一个二叉排序树

```
void CreatBST (BSTree &T) {  
    T = NULL; // 初始化二叉搜索树，根节点设置为NULL  
    cin >> e; // 从标准输入读取一个元素到e  
    while (e.key != ENDFLAG) {  
        // 当读取的元素的关键字不是结束标志时  
        InsertBST(T, e);  
        // 调用InsertBST函数，将当前元素e插入到二叉搜索树T中  
        cin >> e;  
        // 继续从标准输入读取下一个元素到e  
    }  
}
```



假设有 n 个节点，则需要 n 次插入操作，而插入一个节点的算法时间复杂度为 $O(\log_2 n)$
所以创建二叉排序树算法的时间复杂度为 $O(n \log_2 n)$



树形查找

二叉排序树BST
(二叉搜索树)

平衡二叉树AVL

红黑树

查找

$O(\log_2 n)$

插入

$O(\log_2 n)$

删除

$O(\log_2 n)$

创建

$O(n \log_2 n)$



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

平衡二叉树或者是空树，或者是具有以下特征的**二叉排序树**

左子树和右子树的深度之差的绝对值不超过1

左子树和右子树也是平衡二叉树



平衡二叉树或者是空树，或者是具有以下特征的**二叉排序树**

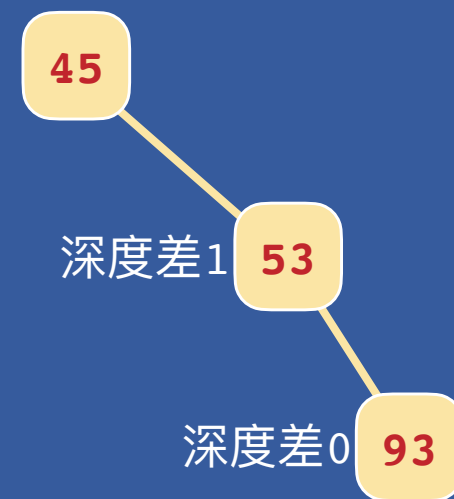
左子树和右子树的深度之差的绝对值不超过1

左子树和右子树也是平衡二叉树

非法的平衡二叉树



深度差2



AVL上的节点的左子树和右子树的深度之差称为平衡因子。

它只能是-1、0、1。

只有一棵BST所有节点的平衡因子的**绝对值**都小于等于1，这棵树才是AVL

(深度差此处指差的绝对值)

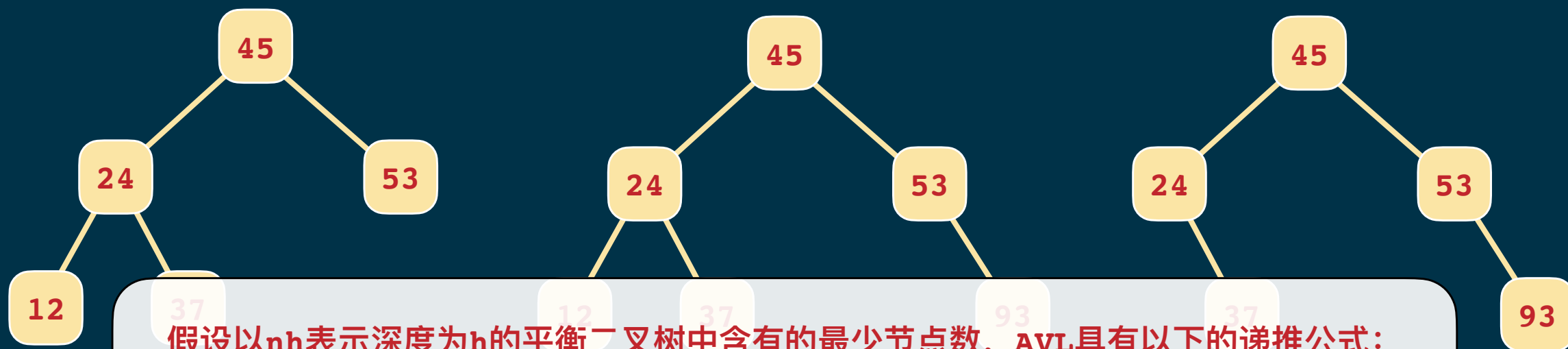


平衡二叉树或者是空树，或者是具有以下特征的二叉排序树

左子树和右子树的深度之差的绝对值不超过1

左子树和右子树也是平衡二叉树

平衡二叉树AVL



假设以 n_h 表示深度为 h 的平衡二叉树中含有的最少节点数，AVL具有以下的递推公式：

$$n_h = n_{h-2} + n_{h-1} + 1$$

显然 $n_0=0, n_1=1, n_2=2$ ，由此可以推出后序所有 n_h



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

查找操作

平衡二叉树AVL是基于二叉排序树BST再加上额外的平衡限制
所以查找操作和BST完全一致



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树



插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

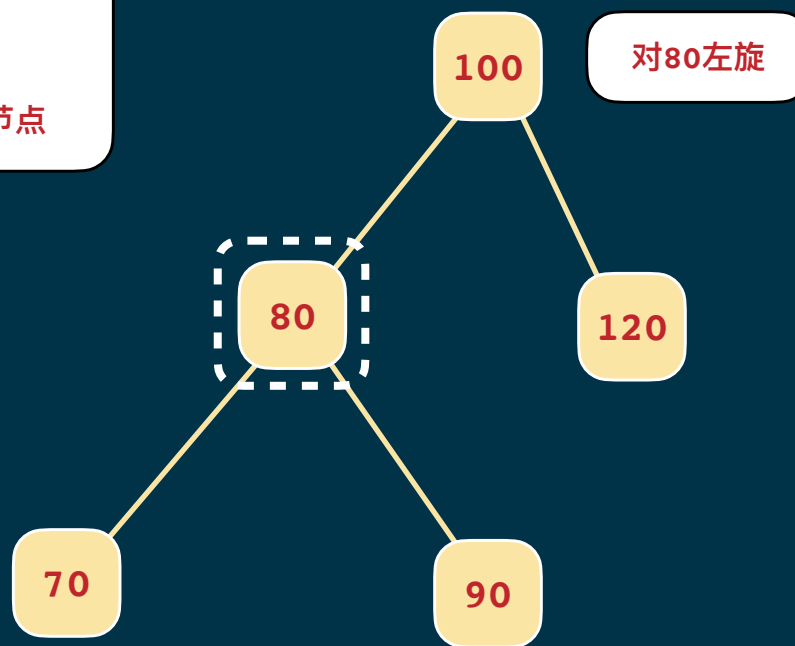
最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

左旋操作：

以某节点为旋转节点

其右子节点变为旋转节点的父节点

右子节点的左子节点变为旋转节点的右子节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

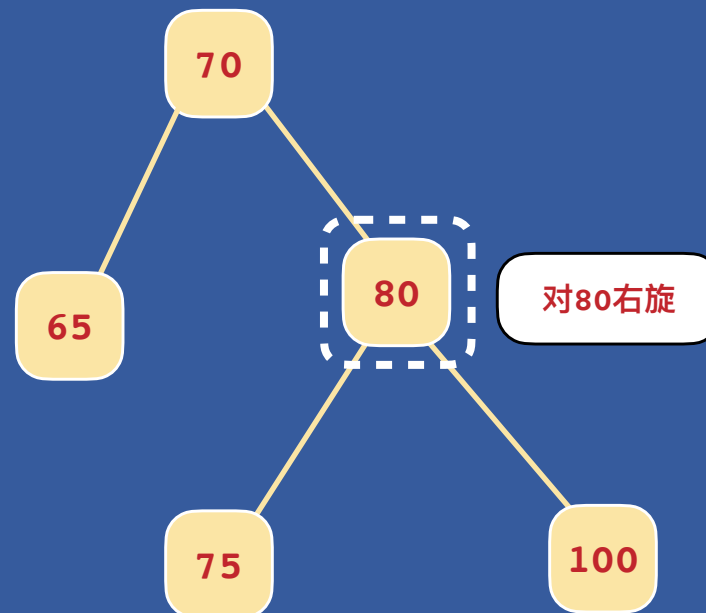
最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

右旋操作：

以某节点为旋转节点

其左子节点变为旋转节点的父节点

左子节点的右子节点变为旋转节点的左子节点





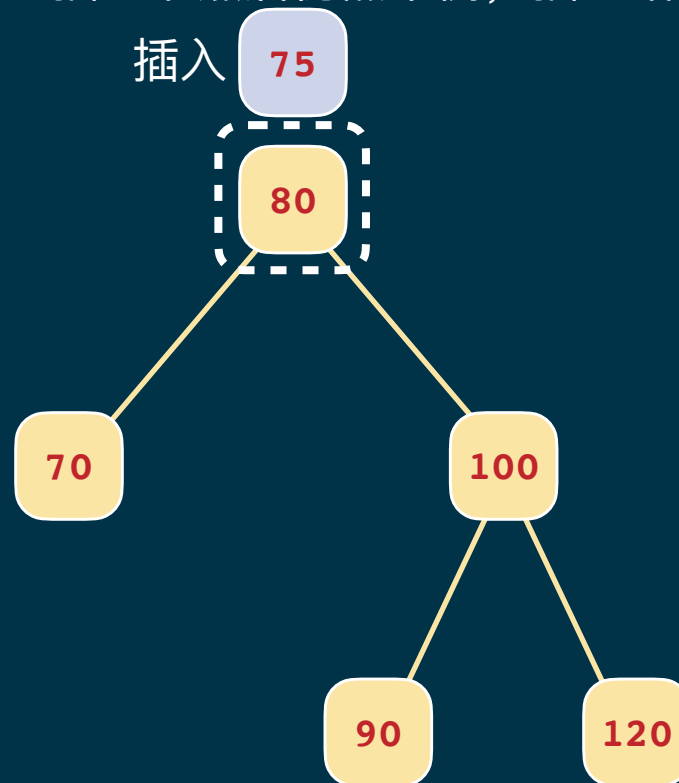
插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，插入结束





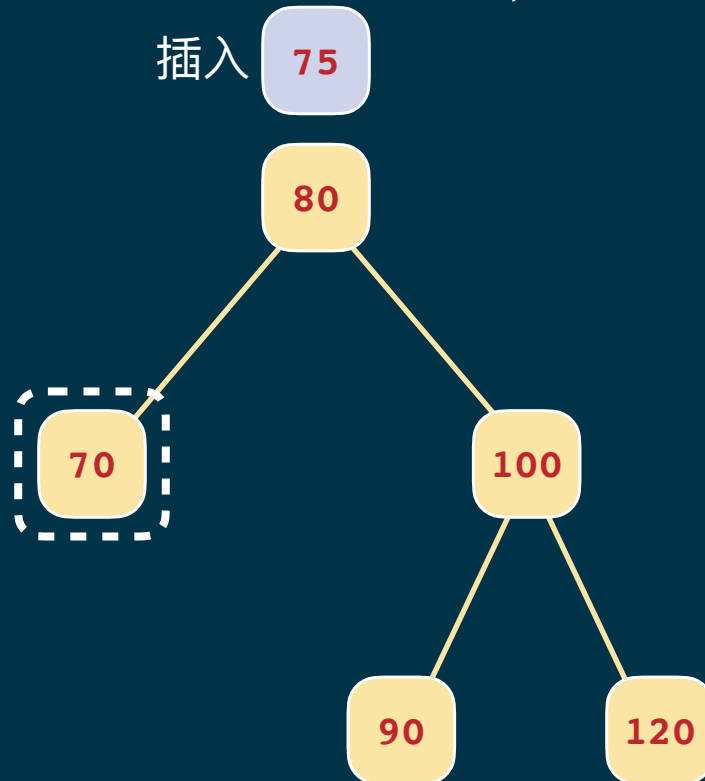
插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，插入结束





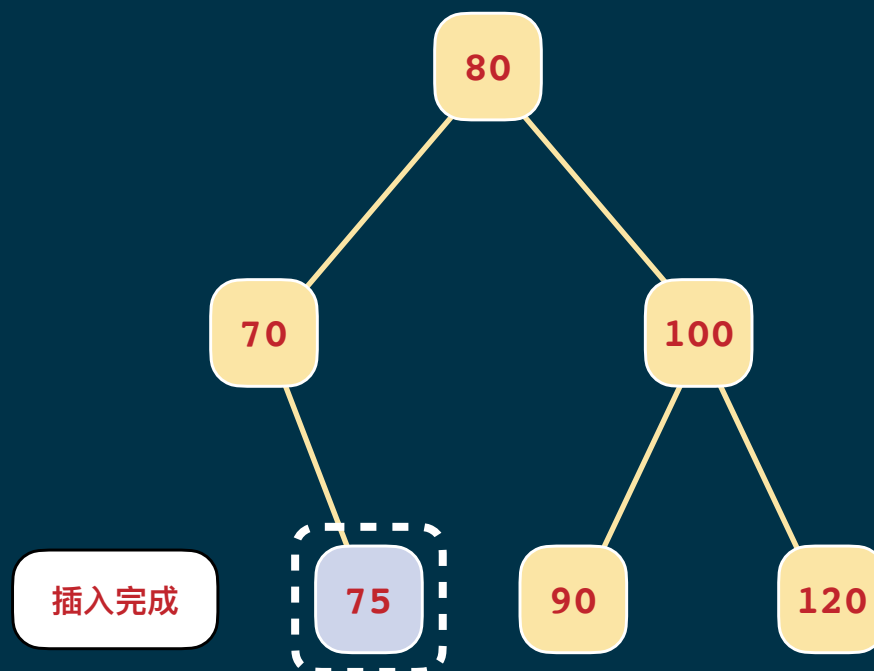
插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，插入结束





插入操作

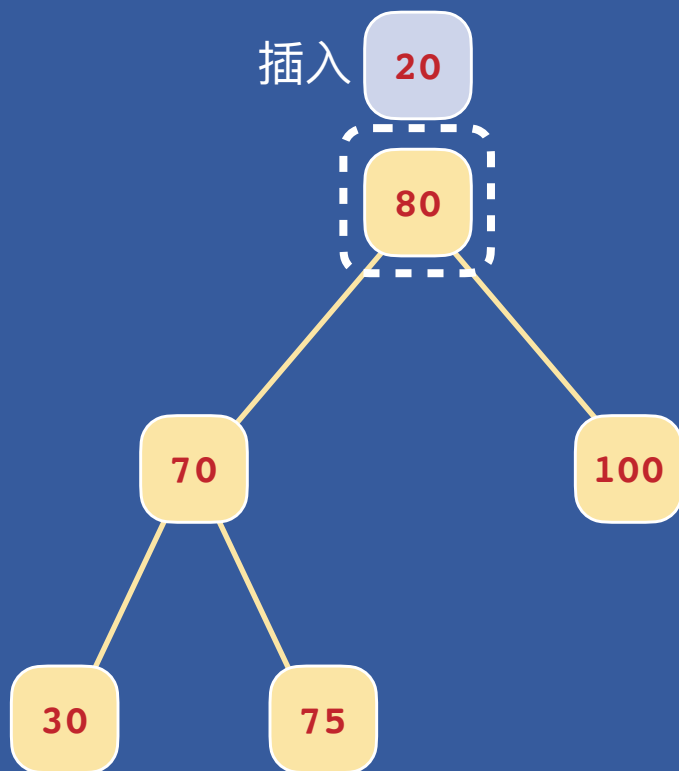
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点





插入操作

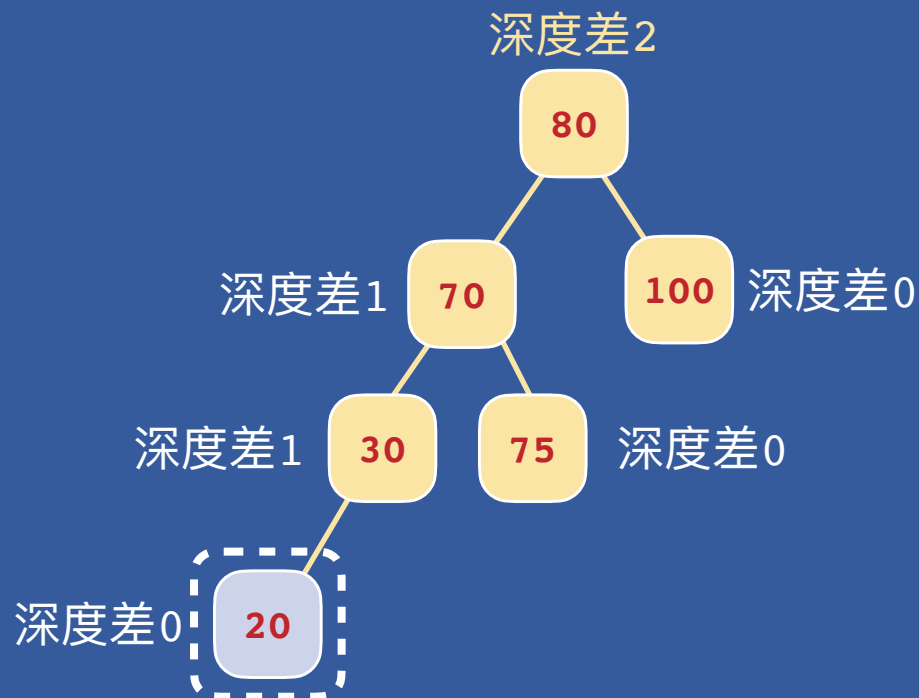
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点



旋转方式：右旋
(与LL字母相反)



插入操作

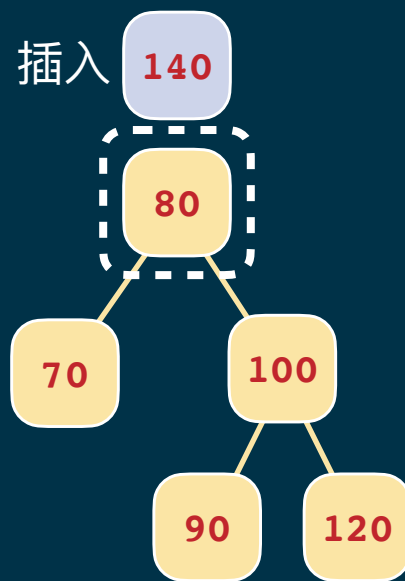
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点





插入操作

插入若不影响平衡，就直接插入

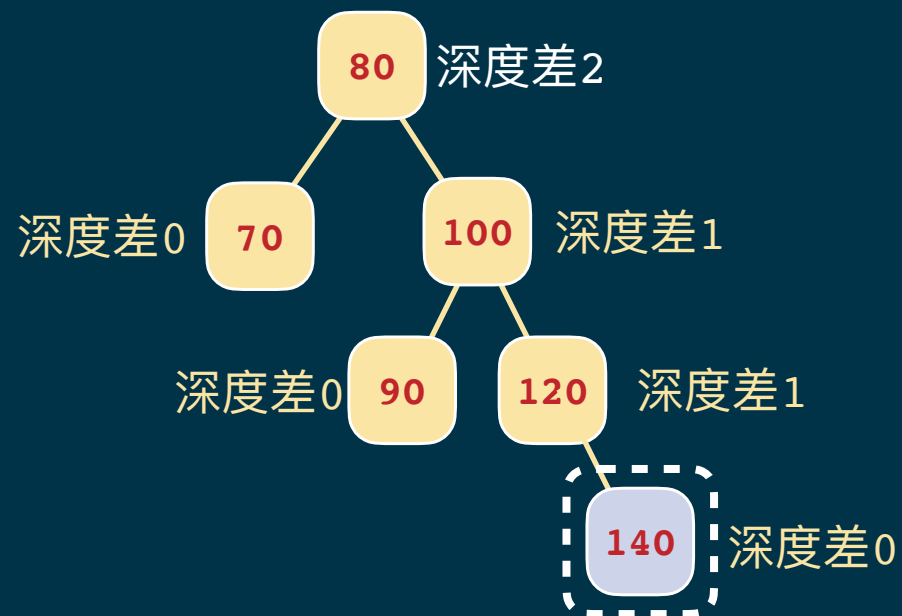
插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点

插入





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点

旋转方式：左旋
(与RR字母相反)

最小不平衡子树根节点

80

深度差2

深度差0

70

100

深度差1

深度差0

90

120

深度差1

140

深度差0



插入操作

插入若不影响平衡，就直接插入

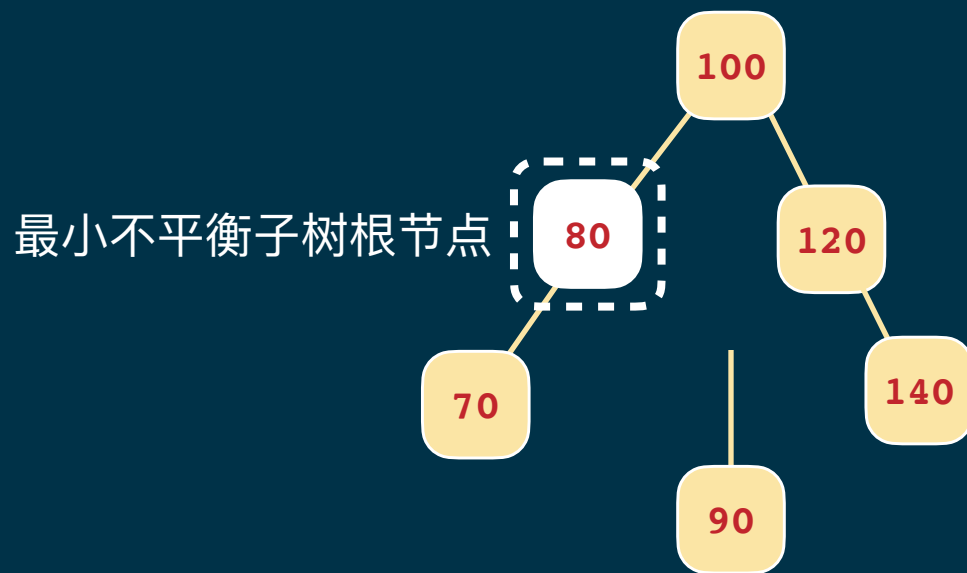
插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点

旋转方式：左旋
(与RR字母相反)





插入操作

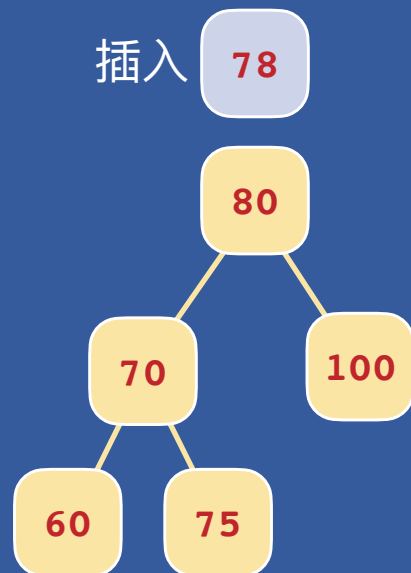
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点





插入操作

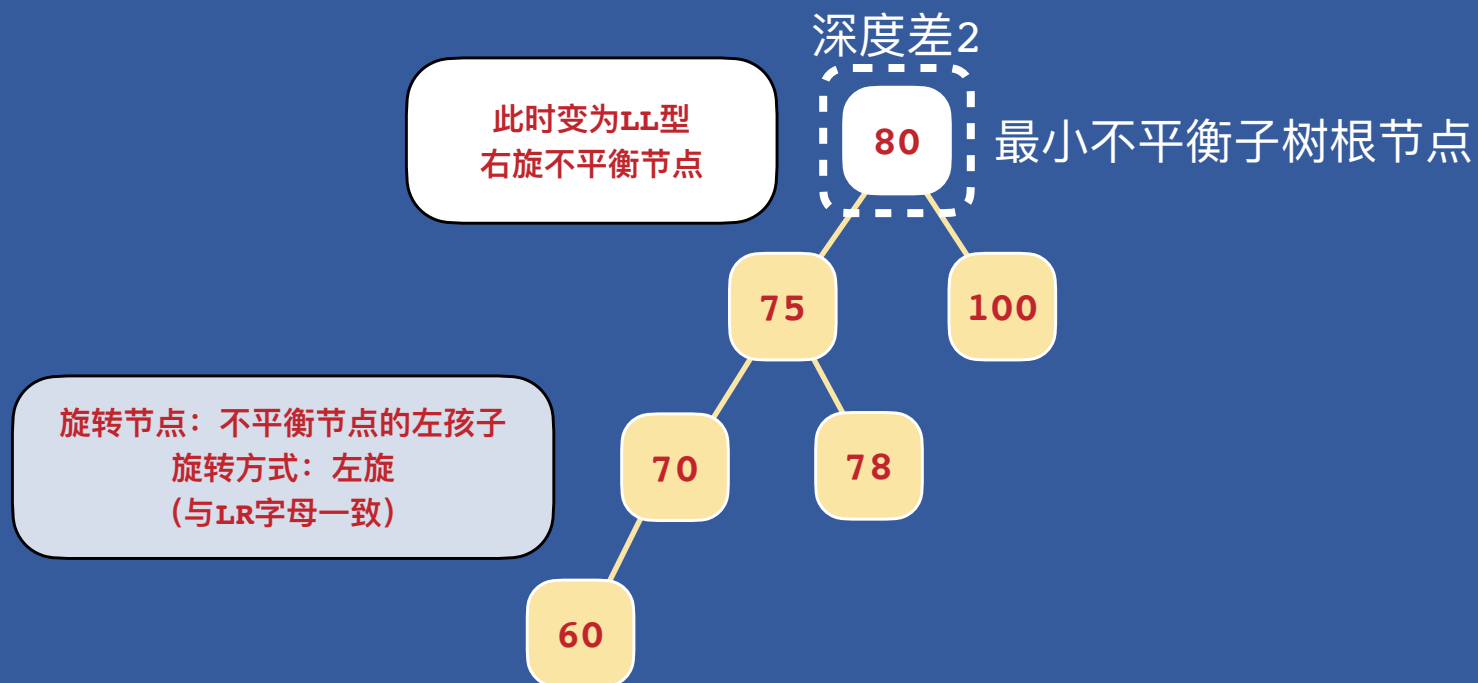
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点





插入操作

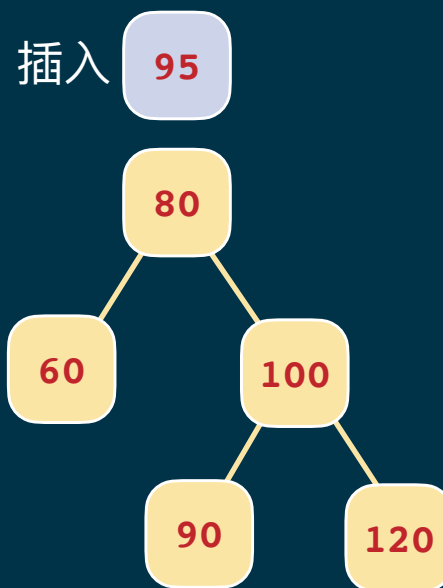
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点





插入操作

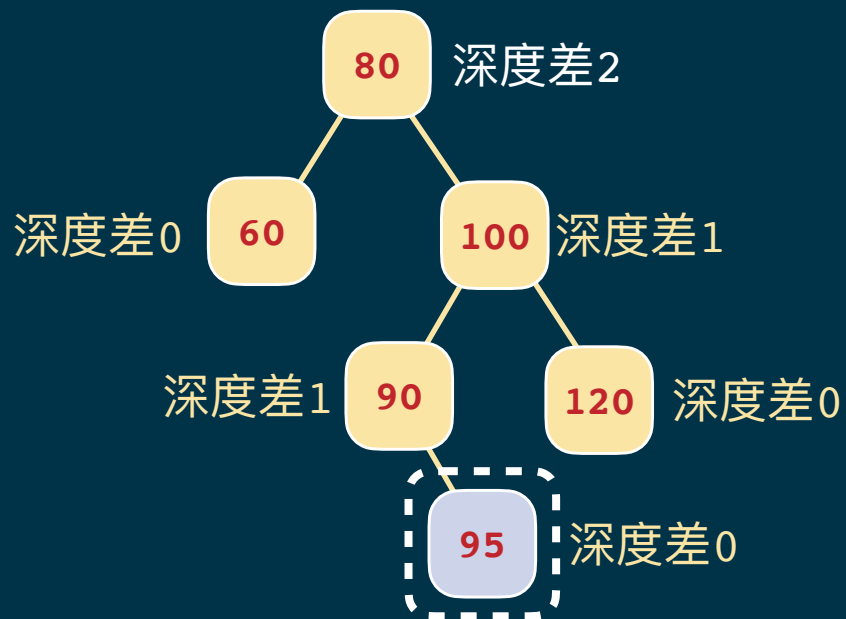
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点





插入操作

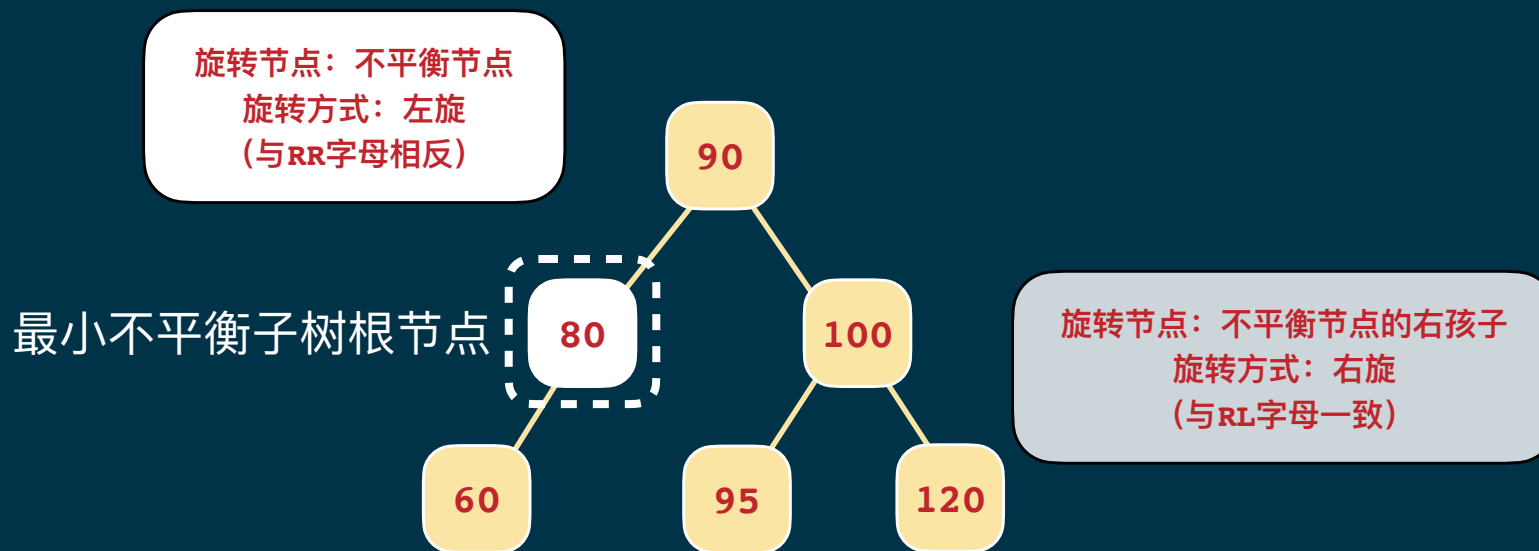
插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况二：插入节点后不平衡

RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点





插入操作

插入若不影响平衡，就直接插入

插入若引起不平衡，则通过调整**最小不平衡子树**完成平衡

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- **LL**：在最小不平衡子树根节点左子树根节点的左子树上插入节点
右旋不平衡节点
- **RR**：在最小不平衡子树根节点右子树根节点的右子树上插入节点
左旋不平衡节点
- **LR**：在最小不平衡子树根节点左子树根节点的右子树上插入节点
先左旋不平衡节点左孩子 再右旋不平衡节点
- **RL**：在最小不平衡子树根节点右子树根节点的左子树上插入节点。
先右旋不平衡节点右孩子 再左旋不平衡节点



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

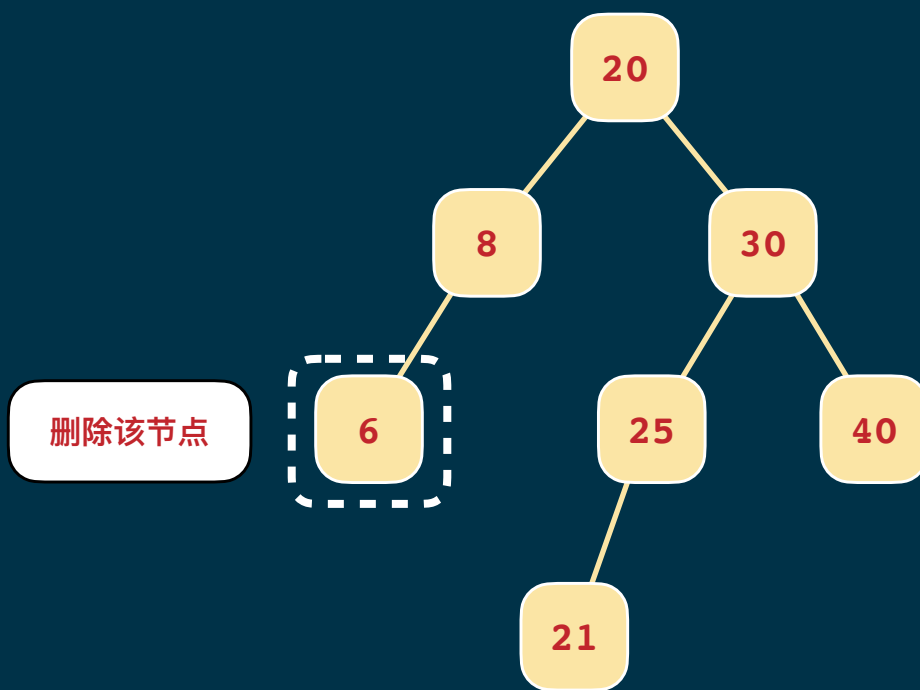


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



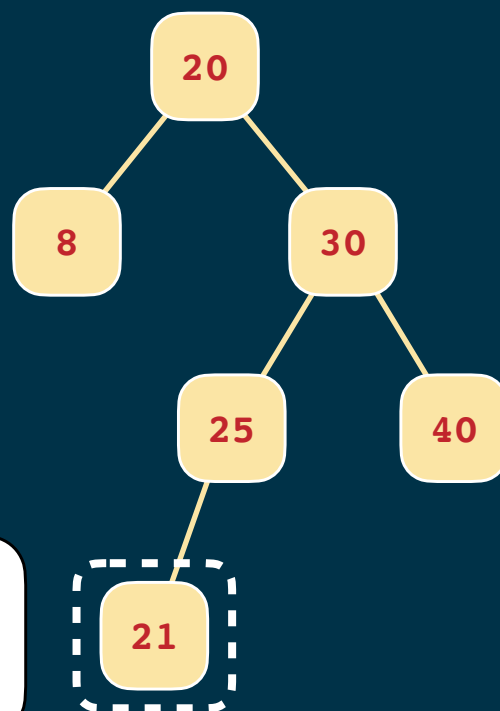


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



转换为插入21的情况 (RL型)
先右旋 (不平衡节点右孩子)
后左旋 (不平衡节点) 解决

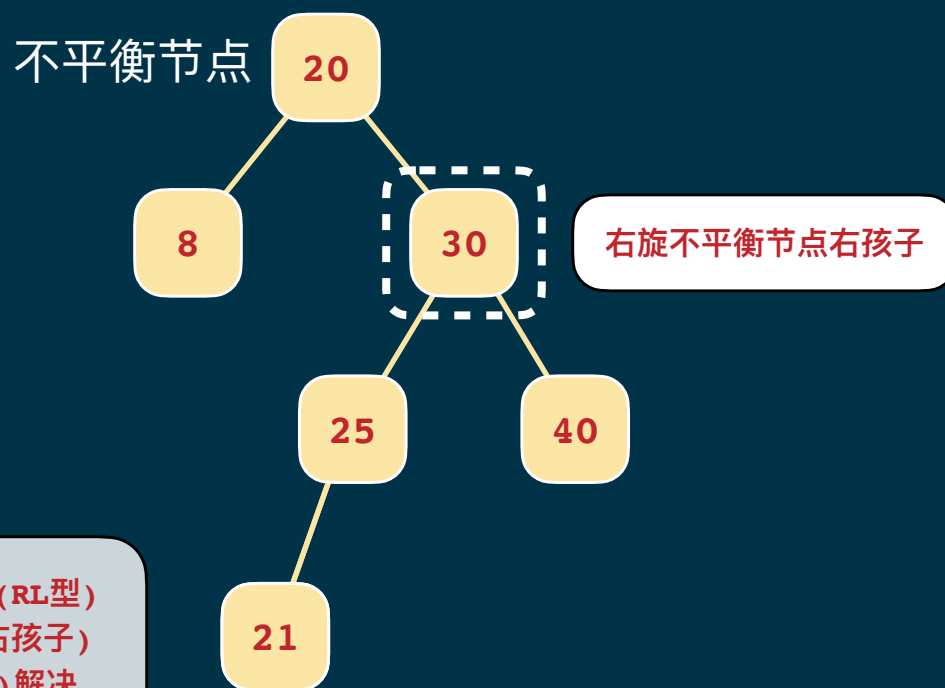


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



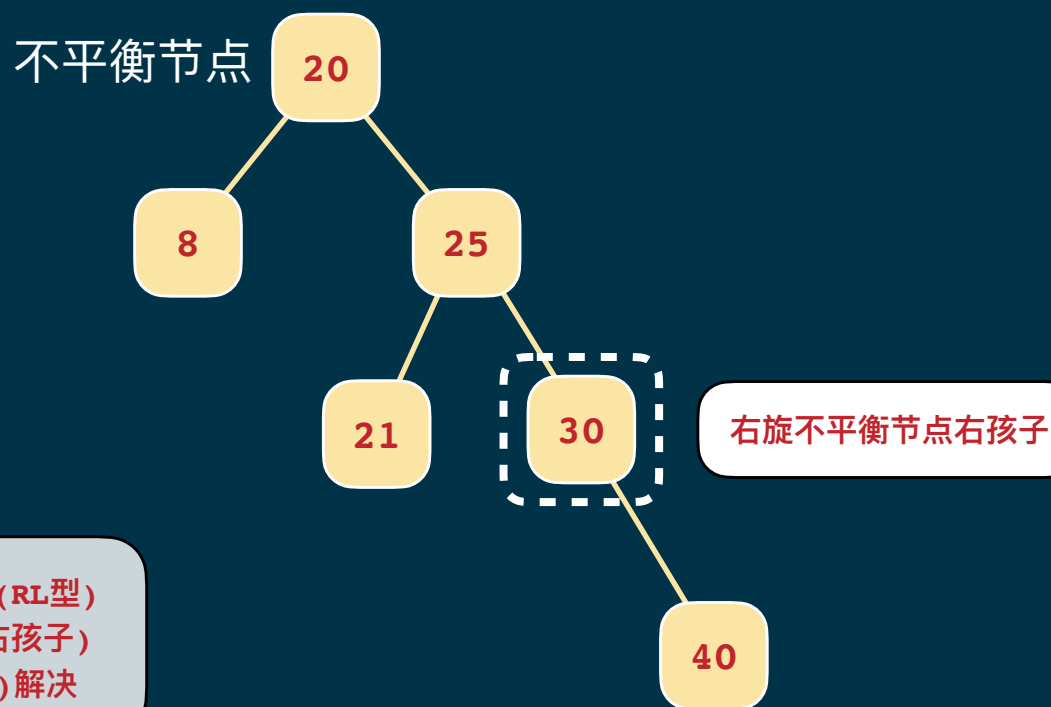


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



转换为插入21的情况(RL型)
先右旋(不平衡节点右孩子)
后左旋(不平衡节点)解决

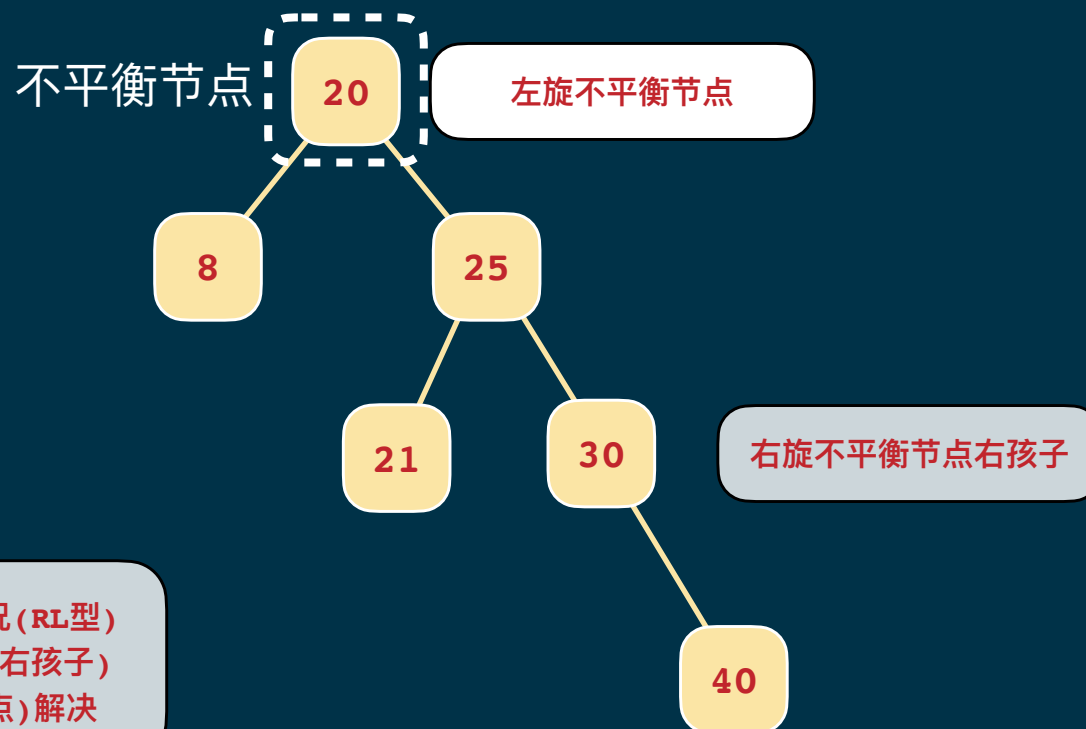


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



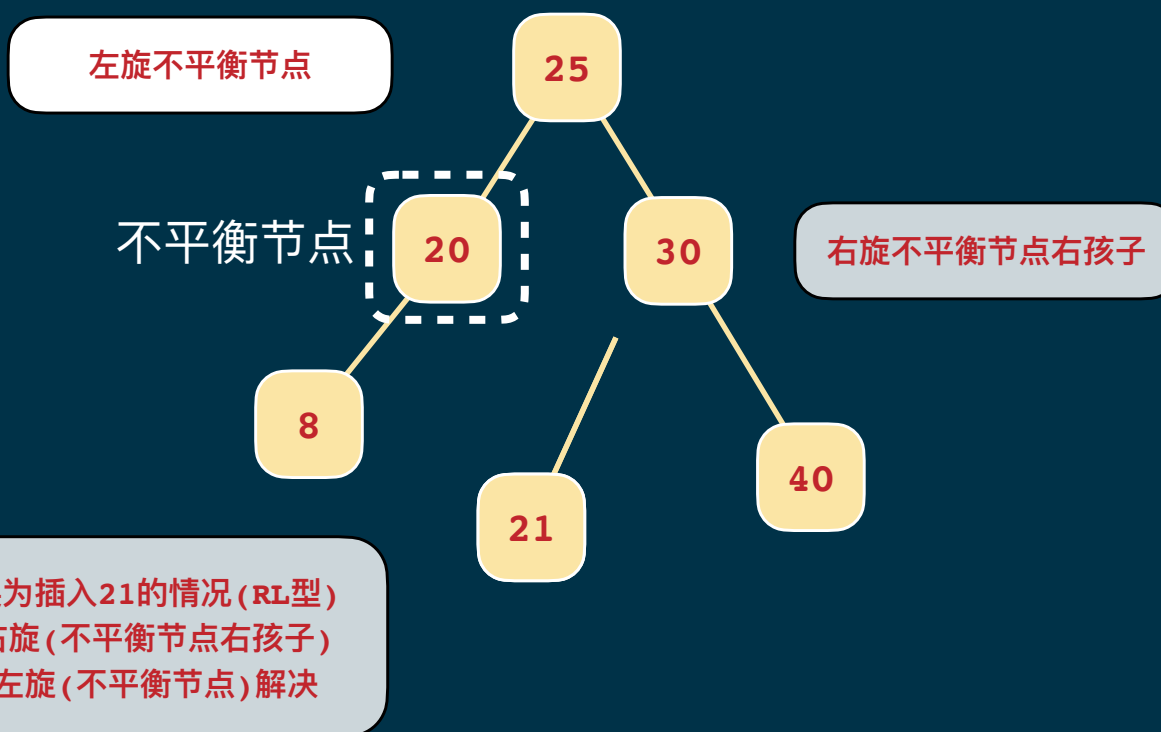


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除叶子节点后不再是平衡二叉树



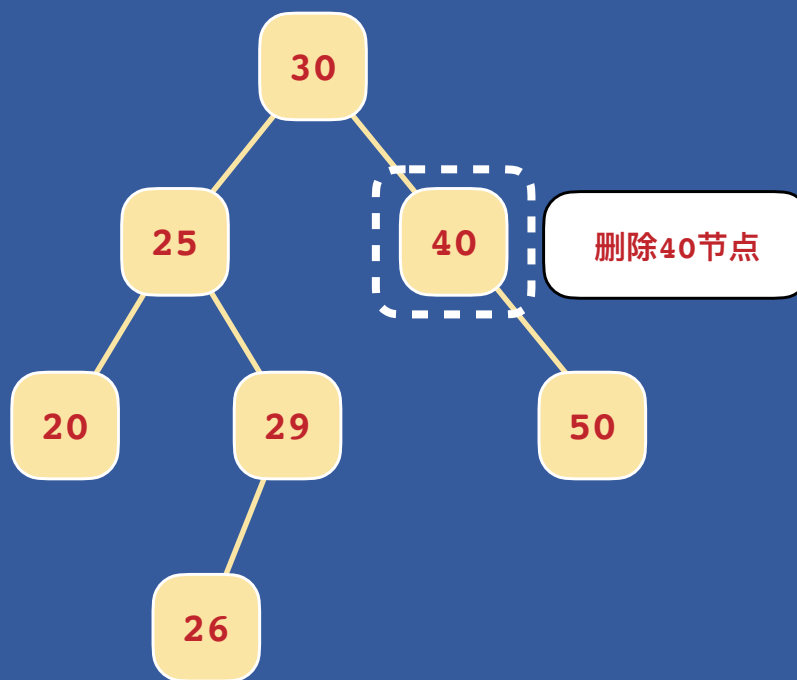


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点只有左/右孩子



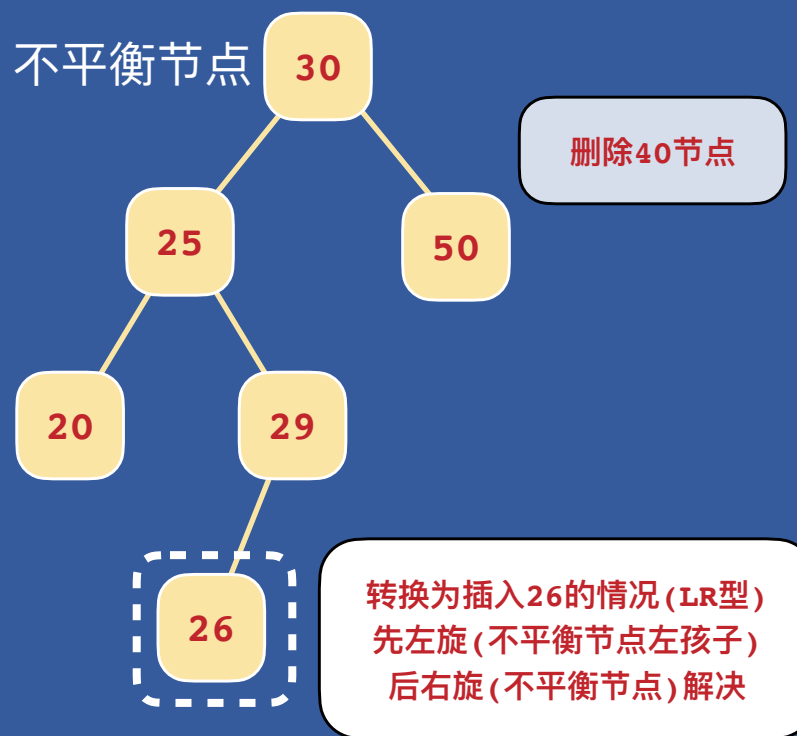


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点只有左/右孩子



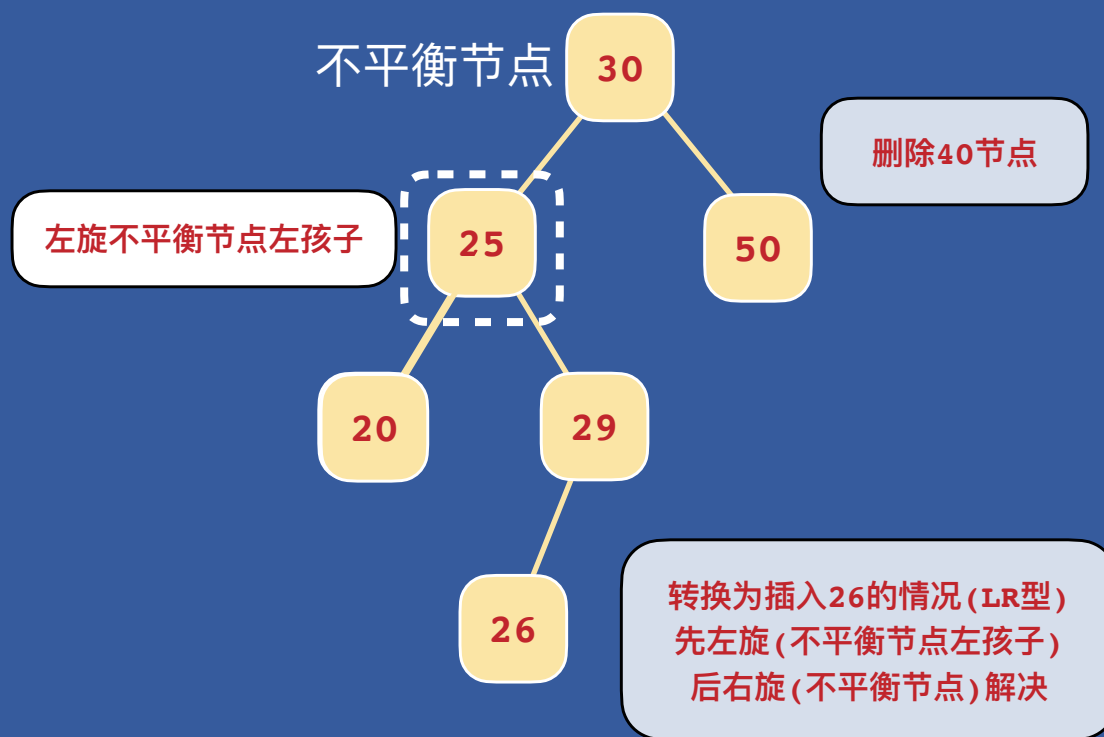


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点只有左/右孩子



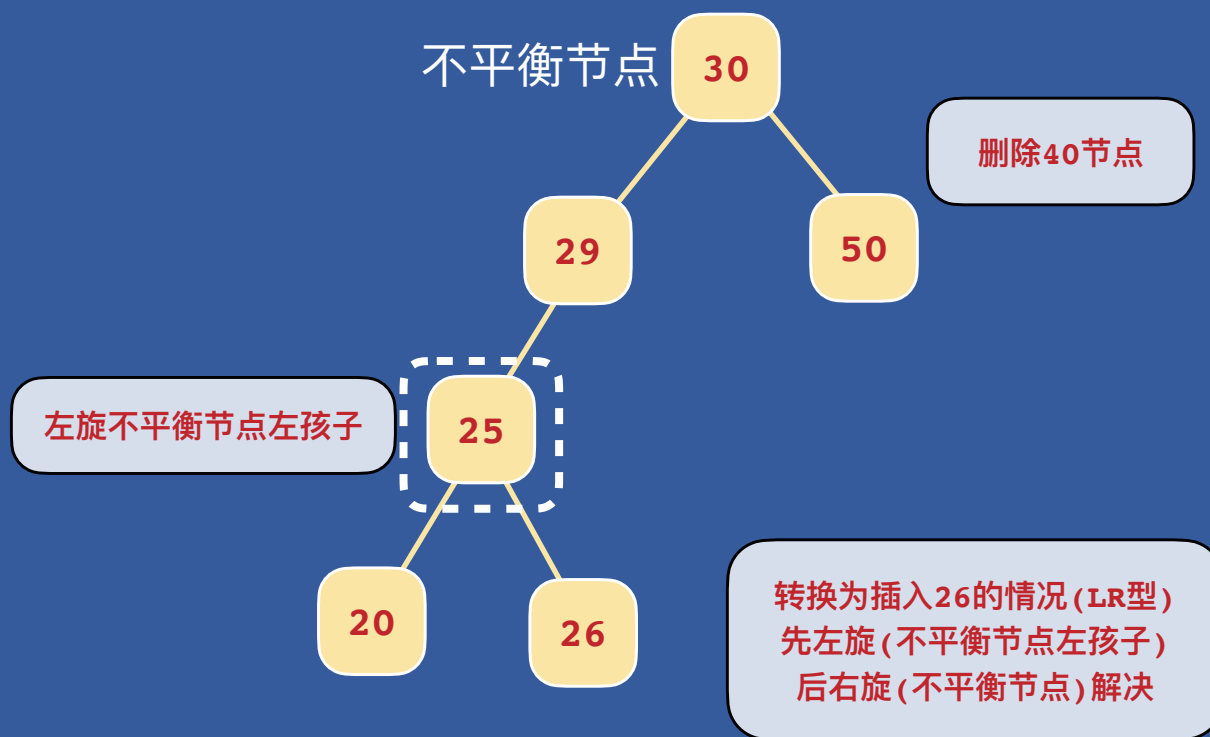


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点只有左/右孩子



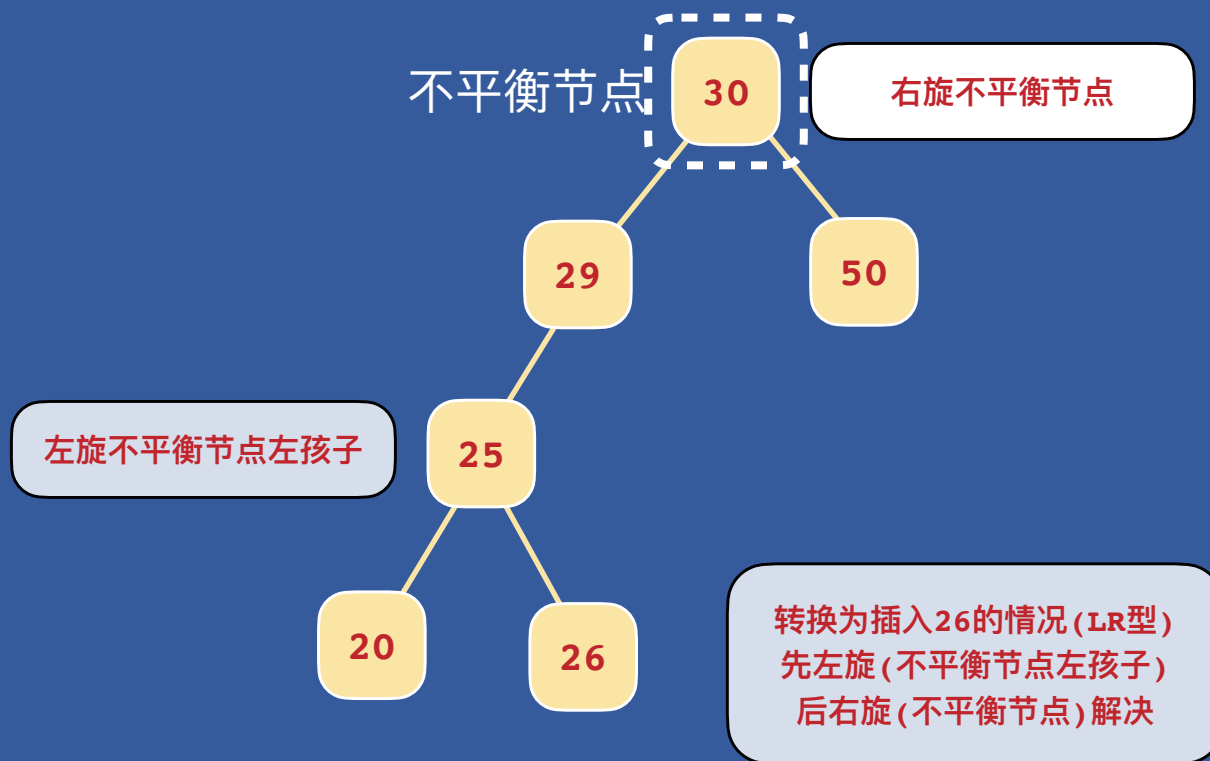


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点只有左/右孩子



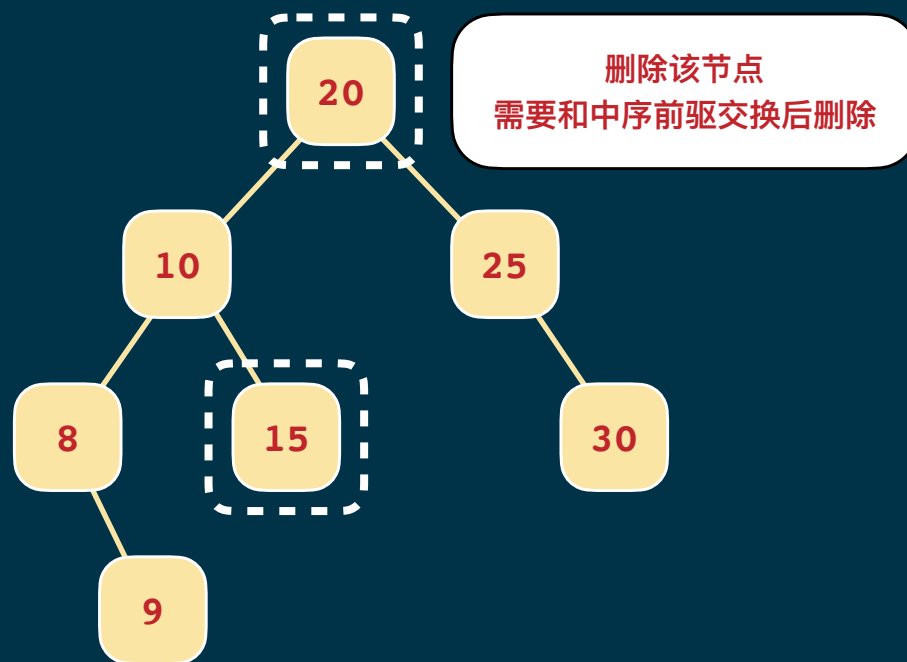


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子为0或1(左边长)



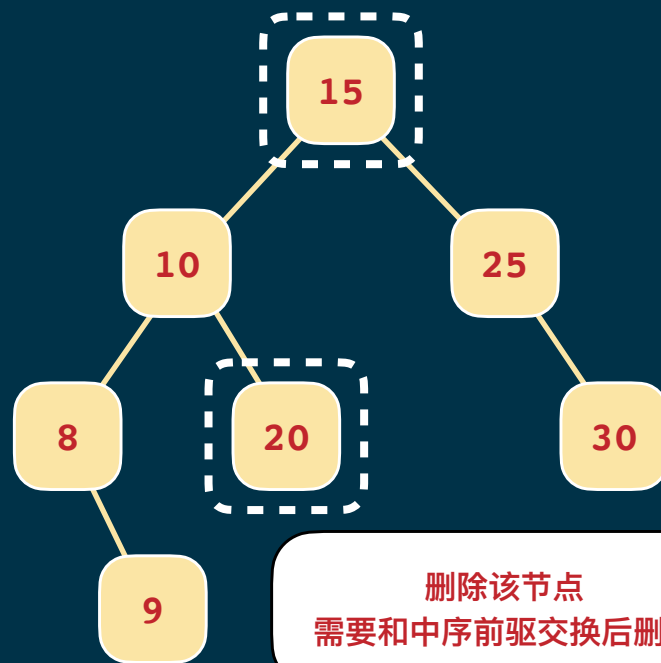


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子为0或1(左边长)



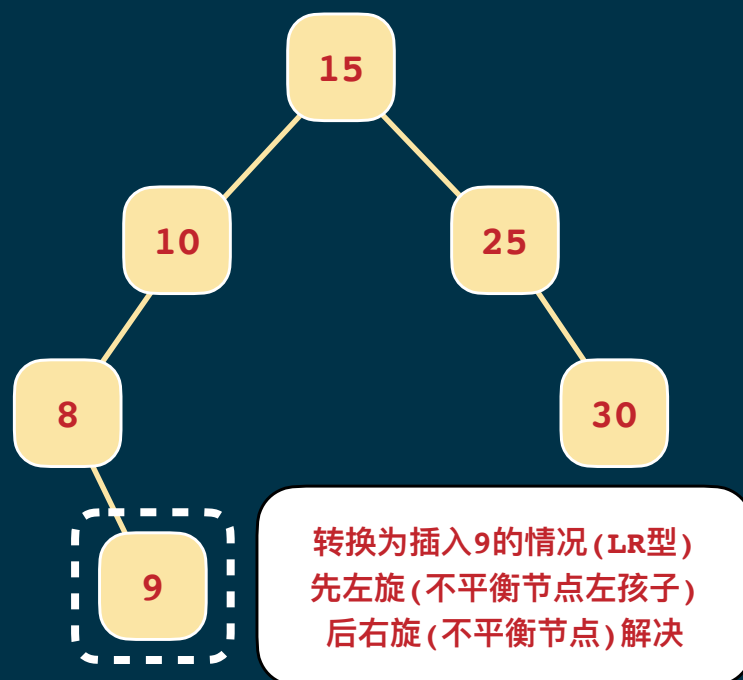


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子为0或1(左边长)



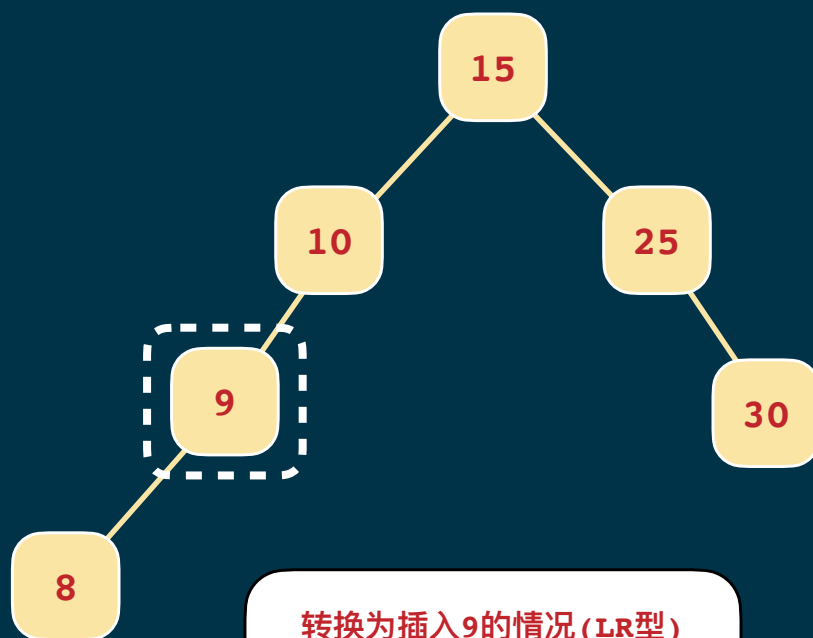


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子为0或1(左边长)



转换为插入9的情况(LR型)
先左旋(不平衡节点左孩子)
后右旋(不平衡节点)解决

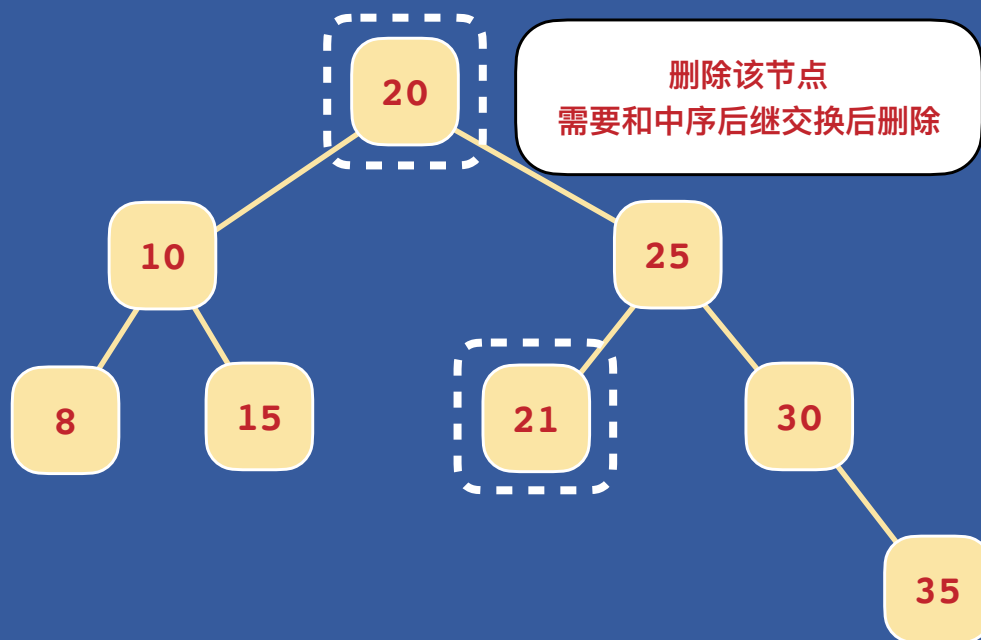


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子=-1(右边长)



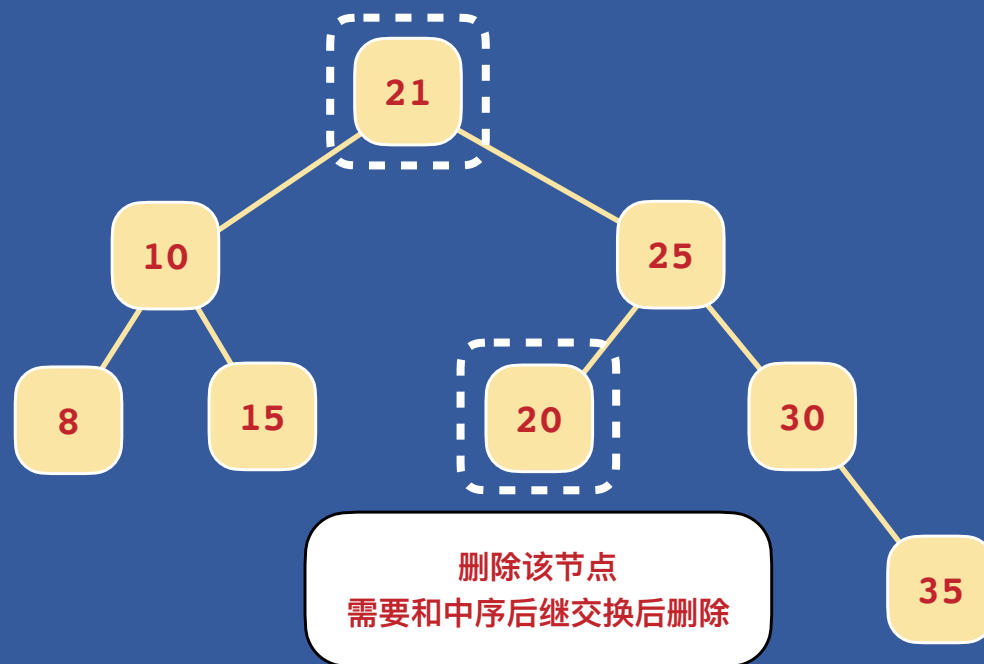


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子=-1(右边长)



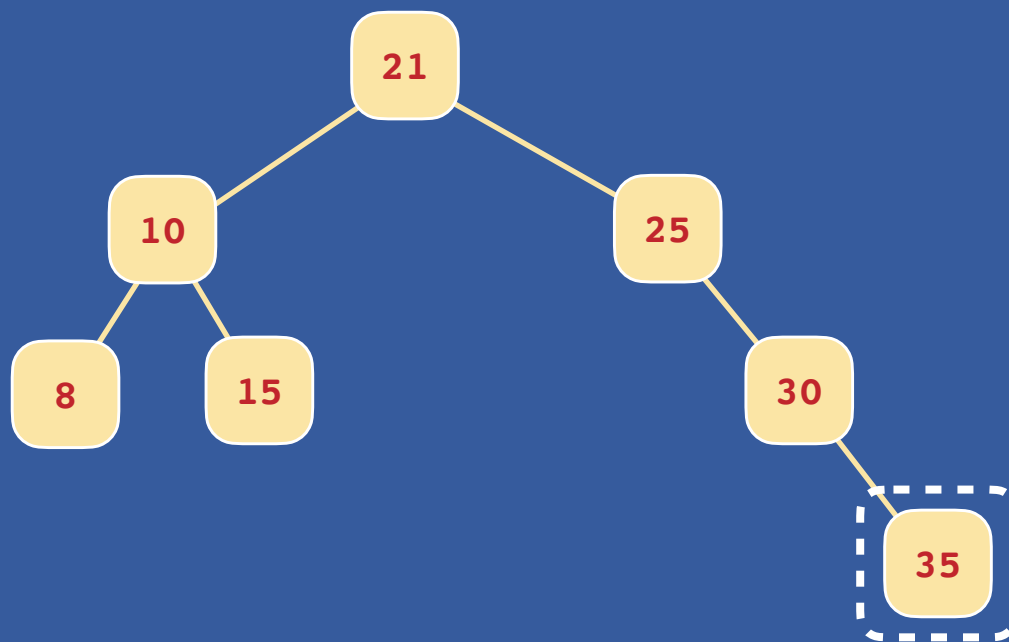


删除操作

删除后若不影响平衡，则直接结束

删除操作若引起不平衡，**转换为插入情况**再进行调整

删除节点同时有左右孩子
删除节点平衡因子=-1(右边长)



等价于插入35
转换为RR型
左旋解决



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

创建操作

平衡二叉树的创建就是不断执行插入操作的过程